Automated Equity Research — Project Guide

Repo: github.com/darrenye1/public-reports- | Site: public-reports-one.vercel.app | Educational demo only; not investment advice.

1. Project Overview

This repository is an end-to-end equity research pipeline. It pulls US mega-cap data from Yahoo Finance (yfinance), runs a five-pillar analysis (valuation, financial-statement analysis, technicals, quantitative metrics, and a composite signal), writes institutional-style 2-page PDF reports plus a Top 20 summary, syncs a static dashboard, and deploys on Vercel.

There is no application server and no React build. Python generates PDFs and JavaScript data files locally or on GitHub Actions; Vercel only hosts the static folder public/.

- Data source: Yahoo Finance via yfinance (delayed; no paid terminal)
- Outputs: reports/{TICKER}_research.pdf, Top20_Summary.pdf / .csv
- Website root: public/ (Vercel Output Directory = public)
- Automation: GitHub Actions every Monday 14:00 UTC, plus manual Run workflow
- State: reports/.data_state.json tracks last run and latest financial period

2. Directory Layout

```
equity_research.py Orchestration, valuation, PDF, batch CLI
market_analytics.py SMA 42/252, RSI, momentum, Sharpe, SMA backtest
financial_statement_analysis.py FSA health score, margin trends, 1Y/3Y projections
signal_blend.py Composite rating + confluence
dashboard_data.py Macro headlines + 6M index sparklines
sync_website.py Copy reports/ into public/ and docs/; write JS
reports/ Generated PDFs/CSV and .data_state.json
public/ Deploy root: index.html, *.js, reports/
docs/ Mirror of public/ (GitHub Pages fallback)
.github/workflows/ weekly-update.yml scheduled refresh
```

3. End-to-End Workflow

Trigger (local CLI or GitHub Actions) → rank MEGA_CAP_UNIVERSE by live market cap → select Top 20 + extra PDF names → fetch Yahoo data → five-pillar analysis → write PDFs → write Top20_Summary.csv → sync_website.py → git push → Vercel redeploy. Pushing code alone does not regenerate PDFs.

3.1 Fully automatic (recommended)

- Monday 14:00 UTC: GitHub Actions cron fires
- python equity_research.py --batch-top20 --refresh-stale --replace-old
- python sync_website.py updates public/ and docs/
- Actions bot commits and pushes main; Vercel redeploys in ~1–2 minutes
- Prerequisite: Settings → Actions → workflow permissions = Read and write

3.2 Local full refresh

```
python equity_research.py --batch-top20 --force --replace-old
python sync_website.py
GitHub Desktop: Commit + Push origin
```

3.3 Single ticker

```
python equity_research.py NVDA
python equity_research.py SPCX --export-excel
```

3.4 Smart refresh (--refresh-stale)

After fetch_company(), confirm_ticker_refresh() reads .data_state.json. A ticker is regenerated if: (a) --force, (b) no prior run, (c) the latest financial-statement period changed, or (d) last run is older than STALE_MAX_AGE_DAYS (default 7). Otherwise the cached summary row is reused and the PDF is left unchanged.

4. How Python Runs One Ticker

All analysis funnels through build_full_analysis() in equity_research.py. process_ticker_report() then calls generate_pdf_report(). process_ticker_summary_only() builds the same analysis without writing a PDF.

```
fetch_company(ticker) # yfinance → CompanyData
build_financial_summary() # price, margins, FCF, analyst target
```

```
build_industry_research() # sector text, trends, risks
build_competitor_analysis() # industry peers + median multiples
build_valuation_summary() # DCF + comps + analyst blend
build_technical_summary() # SMA / RSI / golden-cross
build_quant_summary() # momentum, Sharpe, vs SPY
run_sma_backtest() # SMA strategy vs buy-and-hold
build_financial_statement_summary() # FSA health score
blend_investment_signal() # composite + confluence
generate_pdf_report() # 2-page Letter PDF
```

4.1 Data fetch — fetch_company()

- yf.Ticker: .info, financials, balance_sheet, cashflow, 5Y history, recommendations
- Up to 6 retries with backoff on rate limits / HTTP 429 / timeouts
- Batch mode sleeps ~2s between tickers to reduce Yahoo throttling
- Returns a CompanyData dataclass consumed by every later step

4.2 Universe and batch selection — run_batch()

- MEGA_CAP_UNIVERSE (~90 names) ranked by live US market cap
- build_summary_ticker_list: Top 20 rows; SUMMARY_INCLUDE_TICKERS (SPCX) always included
- build_pdf_ticker_list: Top 20 PDFs plus EXTRA_PDF_TICKERS (TSLA, BX) = 21 reports
- Summary CSV/PDF sorted by market cap

5. Five-Pillar Methodology

The dashboard Composite rating is not the same as the Valuation rating. Valuation produces the target price (45% of the composite). FSA, technicals, and quant supply the remaining 55%. Confluence measures how many pillars point the same way — it is not a statistical confidence interval.

Pillar	Module	Weight	What it measures
Pillar	Module	Composite weight	What it measures
Valuation	equity_research.py	45%	Blended target vs price (DCF / comps / analyst)
FSA	financial_statement_analysis.py	20%	Statement quality: growth, margins, leverage, FCF
Technical	market_analytics.py	20%	SMA 42/252 trend, RSI, golden/death cross
Quantitative	market_analytics.py	15%	Momentum, Sharpe, drawdown, excess vs SPY

5.1 Valuation (45%) — target price engine

build_valuation_summary() (1) picks an industry valuation profile, (2) computes three implied prices, (3) drops unreliable methods, (4) renormalizes profile weights, (5) audits extreme upside, (6) maps upside % to BUY / OVERWEIGHT / HOLD / UNDERWEIGHT / SELL.

5.1.1 Industry profiles — resolve_valuation_profile()

Yahoo industry/sector (and revenue growth) map to a profile that decides whether DCF is used and which comps multiples are allowed. High-growth semiconductors often exclude DCF because FCFE is distorted by heavy capex — that is why a DCF print far below the market can carry 0% weight while comps set the target.

Profile	Typical names	DCF	Comps	Default weights
default	Operating companies	FCFE	P/E, EV/EBITDA, P/S	DCF 32 / Comps 53 / Analyst 15
high_growth_tech	Semis / software, high growth	Off	P/S, EV/EBITDA	Comps 85 / Analyst 15
financial_bank	Banks	Off	P/B	Comps 75 / Analyst 25
financial_insurance	Insurance / AM	Off	P/B	Comps 70 / Analyst 30
financial_conglomerate	BRK-B holdings	Off	Book ~1.0x	Comps 55 / Analyst 45
payment_network	V, MA	FCFE	P/E, EV/EBITDA	DCF 40 / Comps 45 / Analyst 15

5.1.2 FCFE DCF — run_dcf()

Equity-side DCF (not WACC/FCFF). Financials skip DCF entirely.

Cost of equity Ke from CAPM (Yahoo beta). Base FCFE is a robust average of historical FCFE (OCF – capex ± net borrowing); if that series is depressed versus standard FCF, the code falls back to standard FCF.

Five explicit years: FCFE_t = FCFE_{t-1} × (1+g), with g capped (typically 12%, up to 20% for high growth). Mid-year discounting: PV = FCFE_t / (1+Ke)^(t–0.5). Terminal value is Gordon: TV = FCFE_5 × (1+g_term) / (Ke – g_term), then discounted. Implied price = equity value / shares outstanding.

Hard screens: DCF dropped if implied price < 35% of market, > 2.5× market, or more than ~55% below comps.

5.1.3 Comparables — run_comps()

Peers: INDUSTRY_PEERS first (e.g. Semiconductors → AMD, INTC, QCOM, AVGO), then keyword rules, then broad SECTOR_PEERS. Subject ticker is excluded.

Implied prices: peer-median P/E × EPS; EV/EBITDA × EBITDA – net debt; P/S × revenue; P/B × book (banks/holdcos). Mega-caps may add a 52-week range tri-mean as an uplift anchor. Each print is sanity-checked to 0.25×–3.0× current price, then blended into one comps fair value.

5.1.4 Analyst consensus

Yahoo targetMeanPrice. Dropped or down-weighted if upside vs price > 50% or divergence vs comps > 35% (stale consensus). Analyst weight in the blend is capped near 20%.

5.1.5 Blend, audit, and valuation rating

Active methods keep their profile weights, renormalized to 100%. If |target/price – 1| ≥ 30%, _apply_valuation_audit() may switch to a cross-method median, a 30% price cluster, or a ±40% sanity cap when methods disagree. A single coherent anchor is not blindly capped.

Upside mapping (also requires confirming methods for BUY): >20% BUY (if ≥2 methods confirm), >10% OVERWEIGHT, about –5% to +5% HOLD, below –12% UNDERWEIGHT/SELL. This is the dashboard 'Val …' subtext — not the Composite badge.

5.2 Financial statement analysis (20%)

build_financial_statement_summary() scores statement quality on [–1, +1]: 3Y revenue/NI CAGR, gross and operating-margin trend, current ratio, debt/equity, ROE, FCF/net income. Simple projections: revenue grows at a blend of historical CAGR and TTM growth (clipped –5% to +25%); NI = revenue × operating margin × (1 – 21% tax).

Rating: score ≥ +0.25 Strong, ≤ –0.25 Weak, else Stable. Strong is not a bullish price signal — a stock can be FSA Strong and Quant –0.70 at the same time.

5.3 Technical analysis (20%)

SMA 42 vs SMA 252 on daily closes. Golden/death cross scanned over the last 20 sessions. RSI(14) flags overbought (>70) / oversold (<30). Signal is BULLISH if a recent golden cross or short SMA sits >1% above long SMA; BEARISH on the inverse.

run_sma_backtest() compares an SMA crossover strategy with buy-and-hold (0.10% cost per side). If the strategy loses, the technical score is scaled by 0.65 in the composite so weak TA does not dominate.

5.4 Quantitative (15%)

12-month and 3-month momentum, annualized volatility, Sharpe (Rf = 4%), max drawdown, excess return vs SPY. These collapse into quant_score on [–1, +1] and a POSITIVE / NEUTRAL / NEGATIVE label — the dashboard Quant column.

5.5 Composite rating and confluence — signal_blend.py

Each pillar is mapped to [–1, +1]. Valuation score ≈ upside% / 25 (so +25% upside → +1). Technical: golden cross +0.85, BULLISH +0.5, BEARISH –0.5, death cross –0.85. FSA and quant use their native scores.

composite = 0.45×Val + 0.20×FSA + 0.20×TA + 0.15×Quant. Mapped to BUY (≥0.55), OVERWEIGHT (≥0.25), HOLD, UNDERWEIGHT (≤–0.25), SELL (≤–0.55).

Valuation anchor: large negative upside (≤ –15% / –25%) cannot stay HOLD/BUY; large positive upside can lift HOLD to OVERWEIGHT/BUY.

Confluence: count pillars with score > +0.15 (bullish) vs < –0.15 (bearish); confluence = max(bull, bear) / 4 × 100%. Example: three bullish pillars → 75%; two vs two → 50% (typical HOLD).

6. PDF Report Layout

generate_pdf_report() uses ReportLab (Letter). Matplotlib draws the price/SMA chart at 160 DPI so the embed is sharp. Current layout is two pages.

Page 1 — company deep dive

- Header: name, sector, date, latest financial period

- Rating card: composite rating, valuation rating, target, upside, last price
- Investment Highlights (three bullets) and multi-pillar snapshot table
- Investment thesis / executive summary
- Key data grid (market cap, multiples, margins, FCF, ROE, yield)
- Price trend chart with SMA 42 and SMA 252

Page 2 — financials, valuation, peers

- Financial & valuation summary (2 historical years A + 3 forecast years E)
- Technical and quantitative tables side by side
- Valuation cross-check (DCF, blended comps, analyst, blended target) full width
- DCF (FCFE) one-row summary: Ke, terminal g, FCFE growth, equity value, implied, vs current
- Peer comparison plus a one-line industry context note

7. Website and Vercel Dashboard

The live site is a static single-page app. Python never runs on Vercel. GitHub Actions (or a local machine) produces artifacts, commits them into public/, and Vercel serves that folder on every push to main.

7.1 sync_website.py

- Copy *_research.pdf, Top20_Summary.pdf and .csv into public/reports and docs/reports; delete stale PDFs
- Top20_Summary.csv → public/summary-data.js (window.SUMMARY_ROWS + DATA_AS_OF)
- dashboard_data.build_dashboard_payload() → dashboard-data.js
- Copy index.html and social assets into docs/
- Do not hand-edit the JS data files — change Python and re-run sync

7.2 dashboard_data.py

- News: Yahoo stories on SPY, indices, VIX, dollar, gold, sector ETFs
- Impact score = macro keywords × 3 + premium source points + high-impact terms
- One headline per US Eastern calendar day for the past 7 days
- Index charts: ^GSPC, ^DJI, ^IXIC, ^RUT — 6M closes normalized to 100

7.3 public/index.html

- No npm / bundler. Loads summary-data.js and dashboard-data.js
- Sortable Top 20 table: Composite, TA, FSA, Quant, Target/Upside, PDF link
- KPI strip, methodology cards, 6M sparkline canvases, macro news list
- Portfolio sections: #about #project #research #markets #top20 #resume
- Vercel Web Analytics script (/_vercel/insights/script.js) — enable in the Vercel UI

7.4 Vercel setup

- Connect GitHub repo; Framework = Other / static
- Output Directory = public (root of the repo, not a build folder)
- Each push to main publishes HTML, JS, and PDFs on the CDN
- Visitor stats: Vercel → Analytics → enable Web Analytics (see ANALYTICS.md)

8. GitHub Actions

File: .github/workflows/weekly-update.yml. Triggers: cron 0 14 * * 1 (Monday) and workflow_dispatch. Job: checkout → Python 3.12 → pip install → stale batch (or force-full if the dispatch checkbox is on) → sync_website.py → git add reports/ public/ docs/ → commit → push. Vercel then redeploys.

9. Git Workflow Tips

- Commit source (.py, index.html, workflows). Prefer regenerating PDFs/JS via Actions or local batch + sync
- PDFs are binary: never merge them in VS Code. Keep one side, then re-run the batch
- On summary-data.js / dashboard-data.js conflicts: pick either side, then sync_website.py
- After Actions pushes, Pull before local edits so branches do not diverge
- resolve_merge.bat / resolve_merge.sh keep --ours on generated PDFs and JS during a merge

10. Common Operations

Scenario	Steps
Change valuation or PDF layout	Edit equity_research.py → force batch → sync → push
Change TA / quant / FSA / composite	Edit the matching .py → force batch → sync → push
Add a forced Top 20 ticker	SUMMARY_INCLUDE_TICKERS and MEGA_CAP_UNIVERSE
Add an extra PDF	EXTRA_PDF_TICKERS
Change news / index widgets	Edit dashboard_data.py → python sync_website.py
Change website UI	Edit public/index.html → push (sync copies it to docs/)
Refresh the live site now	GitHub → Actions → Weekly research update → Run workflow
Single-name PDF only	python equity_research.py TICKER

11. Key Constants

Constant	Value	Meaning
TOP_US_SUMMARY_COUNT	20	Summary table rows
EXTRA_PDF_TICKERS	TSLA, BX	Extra PDFs beyond Top 20
SUMMARY_INCLUDE_TICKERS	SPCX	Always in the Top 20 table
STALE_MAX_AGE_DAYS	7	Smart-refresh age threshold
VALUATION_AUDIT_THRESHOLD_PCT	30	Audit trigger |upside| %
TARGET_SANITY_CAP_PCT	40	Hard cap vs current when methods disagree
WEIGHT_FUNDAMENTAL / FSA / TA / Q	45 / 20 / 20 / 15	Composite blend
SMA_SHORT / SMA_LONG	42 / 252	Technical moving averages
ANALYST_MAX_BLEND_WEIGHT	20%	Cap on analyst in valuation blend

12. CLI Reference

```
python equity_research.py NVDA
python equity_research.py --batch-top20 --refresh-stale
python equity_research.py --batch-top20 --force --replace-old
python equity_research.py --batch-top20 --weekly
python sync_website.py
python generate_project_guide_pdf.py
```

13. Dependencies and Disclaimer

Python packages: yfinance, pandas, numpy, reportlab, matplotlib, openpyxl (see requirements.txt).

Regenerate this guide: python generate_project_guide_pdf.py (writes PROJECT_GUIDE.pdf and Equity Research PROJECT_GUIDE.pdf).

Educational / research demo only. Not investment advice. Yahoo Finance data can be delayed, incomplete, or restated; model outputs are mechanical and can be wrong for individual names.